偏微分方程式の数値解析

1424020 海野　颯

2025 年 7 月 16 日

目次

1 **熱伝導方程式の背景や導出について** 2
1.1 背景 . . . 2
1.2 導出 . . . 2

2 **解や誤差の様子について** 3
2.1 $\alpha = 1,\ f(x) = 1,\ a = 0,\ b = 0$ のとき . . . 3
2.2 初期条件の変化に伴う解の様子 . . . 4
2.3 拡散係数 α による変化 . . . 10

3 **反復式について** 12
3.1 偏微分項の追加 . . . 12
3.2 ノイマン境界条件 . . . 14

4 **陰解法について** 15

5 **まとめ** 17

6 **参考文献** 17

7 **MATLAB のコード** 17

概要

定数 $\alpha > 0$ に対して，

$$\begin{cases} \dfrac{\partial u}{\partial t} = \alpha \dfrac{\partial^2 u}{\partial x^2}, & 0 < x < 1,\ 0 < t \leq 1, \\ u(x,0) = f(x), & 0 < x < 1, \\ u(0,t) = a, \quad u(1,t) = b, & 0 < t \leq 1. \end{cases}$$

この一次元熱伝導方程式は，熱の拡散を表す代表的な偏微分方程式であり，工学や自然科学のさまざまな分野に応用されている．本レポートでは，数値的手法を用いてこの方程式を解析することで，物理現象の理解を深め，また数値解法の精度や安定性を考察することを目的とする．

1 熱伝導方程式の背景や導出について

1.1 背景

この方程式はフランスの数学者ジョセフ・フーリエ（Joseph Fourier, 1768 – 1830）によって導かれた．1822年に『熱の解析的理論』で熱方程式を初めて体系的に発表し，熱の伝導が「空間的な温度の 2 階微分」に比例することを提案した．さまざまな分野の使われていて，地球温暖化や気象モデルなどにも使われている．

1.2 導出

この方程式はフーリエの法則（熱流束の定義）とエネルギー保存則から導かれる．

1.2.1 フーリエの法則

まず、熱の流れ（単位時間あたりの単位面積を通る熱量）を定義する．

$$q(x,t) = -k \frac{\partial u}{\partial x}(x,t)$$

- 温度勾配 $\frac{\partial u}{\partial x}$ が大きいほど，より多くの熱が流れる．
- マイナス符号は，熱が「高温 $\to$ 低温」へ流れることを示す．

1.2.2 エネルギー保存則

長さ Δx の小区間 $[x,\, x + \Delta x]$ を考え，この区間内の熱エネルギーの変化を考える．

- 「区間内の熱エネルギーの変化」＝「流入する熱量」 −「流出する熱量」

$$\text{「内部エネルギーの変化」} = \rho c \Delta x \frac{\partial u}{\partial t} \quad (\rho : \text{密度}, c : \text{比熱})$$

$$\text{「出入りする熱量」} = q(x,t) - q(x+\Delta x, t) \approx -\frac{\partial q}{\partial x}\Delta x \quad (q : \text{熱流束})$$

したがって，

$$\rho c \Delta x \frac{\partial u}{\partial t} = -\frac{\partial q}{\partial x}\Delta x \quad \Rightarrow \quad \rho c \frac{\partial u}{\partial t} = -\frac{\partial q}{\partial x}$$

1.2.3　熱伝導方程式の導出

フーリエの法則より，

$$q = -k\frac{\partial u}{\partial x}$$

であるから，

$$\rho c\frac{\partial u}{\partial t} = -\frac{\partial q}{\partial x}$$

に代入すると，

$$\begin{aligned}\rho c\frac{\partial u}{\partial t} &= -\frac{\partial}{\partial x}\left(-k\frac{\partial u}{\partial x}\right)\\ &= k\frac{\partial^2 u}{\partial x^2}\end{aligned}$$

熱拡散係数 α を次のように定義すれば，

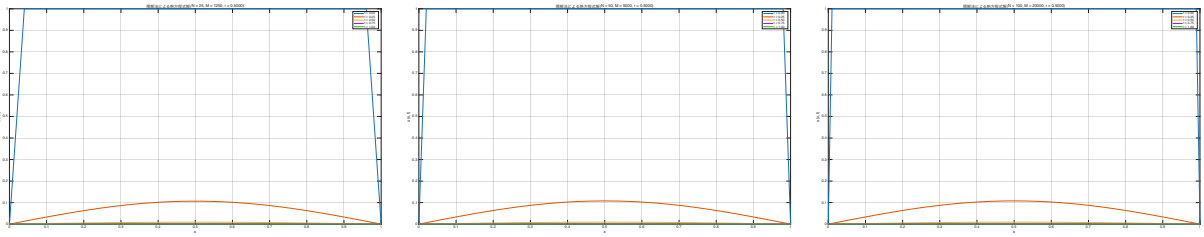
$$\alpha = \frac{k}{\rho c}$$

$$\frac{\partial u}{\partial t} = \alpha\frac{\partial^2 u}{\partial x^2} \quad \square$$

2　解や誤差の様子について

2.1　$\alpha = 1$，$f(x) = 1$，$a = 0$，$b = 0$ のとき

ここでは陽解法を用いて，試しにこの条件で数値解の様子，誤差伝播を見たいと思う．空間分割数を N，時間分割数 M として，$(N, M) = (25, 1250), (50, 5000), (100, 20000)$ のように変化させて，各時間 t ごとに陽解法により得られる解 $u(x, t)$ の様子，誤差伝播の様子を見る．



$(N, M) = (25, 1250)$　　$(N, M) = (50, 5000)$　　$(N, M) = (100, 20000)$

図 1: 各 (N, M) に対する数値解 u の様子

図 1 は分割数 N, M で時刻 t を $t = 0.00, 0.25, 0.50, 0.75, 1.00$ と変化させた様子である．各グラフであまり違いが見られないが，$t = 0$ は初期状態 $(f(x) = 1)$ を表しており，端点 $(x = 0, 1)$ 付近では，分割数の増加に伴って傾きがが急になっている．また，時刻 t が増える（時間が経つ）と，お椀型を保ちながら解は 0 に近づき，初期条件が $f(x) = 1$ という定数関数のため，境界 $u(0, t) = 0, u(1, t) = 0$ の影響が徐々に中心に伝わるだけの単調な減衰である．

次に，誤差を見るために，高精度解として $(N, M) = (400, 320000)$ 設定し，誤差を見る．空間 x に対する誤差の大きさのグラフを以下に示す．

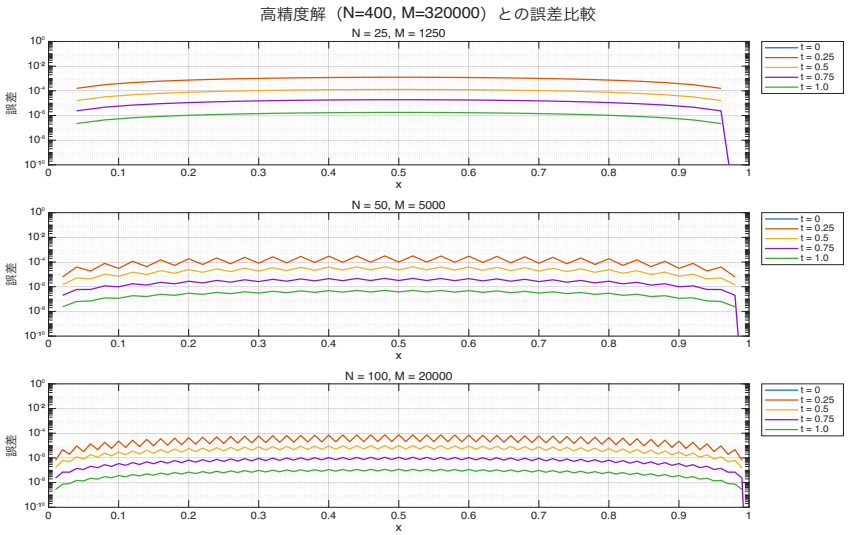


図 2: 各 (N, M) について，空間 x に対する絶対誤差

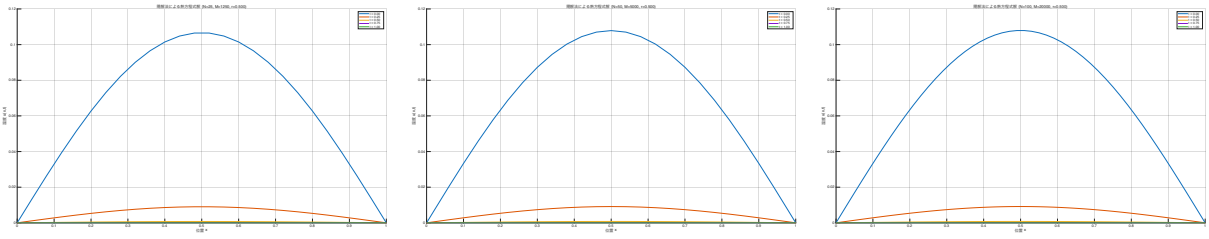
図 2 から，絶対誤差については，分割数 $(N, M) = (25, 1250)$ のとき，絶対誤差は 10^{-1} から 10^{-3} 程度とかなり大きいが，$(N, M) = (100, 2000)$ まで増やすと絶対誤差は 10^{-4} から 10^{-8} 程度まで抑えられており，後半は機械的な丸め誤差レベルである．このことから，分割数が増えるほど陽解法による数値解が基準解に近づいていることが分かる．さらに，周期的に山と谷のようなパターンが見られ，これは数値解法に伴う振動的な誤差伝播の特徴であると考えられる．ただし，$x = 1$ で誤差が急激に落ちているのは，$f(x) = 1$ は $x = 1$ で 0 まで落ちるからであることがわかる．

2.2 初期条件の変化に伴う解の様子

様々な初期条件に対して，分割数による解の様子，誤差伝播の様子を見る．ただし，拡散係数 $\alpha = 1$ とし，$a = b = 0$ とする．

2.2.1 初期条件 $f(x)$ が $0 \leq x \leq 1$ で連続な場合

まず $f(x) = x - x^2$ の場合について考察する．



$(N, M) = (25, 1250)$　　$(N, M) = (50, 5000)$　　$(N, M) = (100, 20000)$

図 3: 各 (N, M) に対する数値解 u の様子

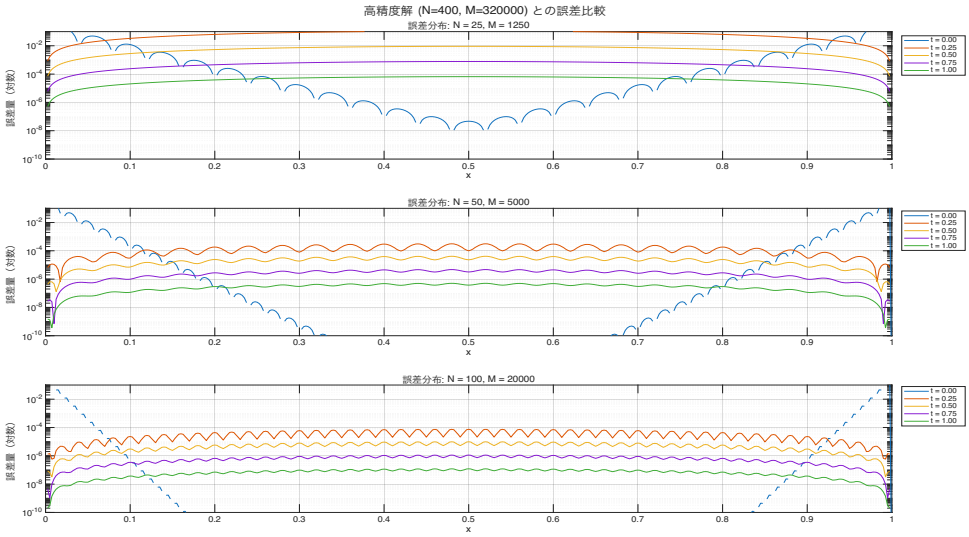


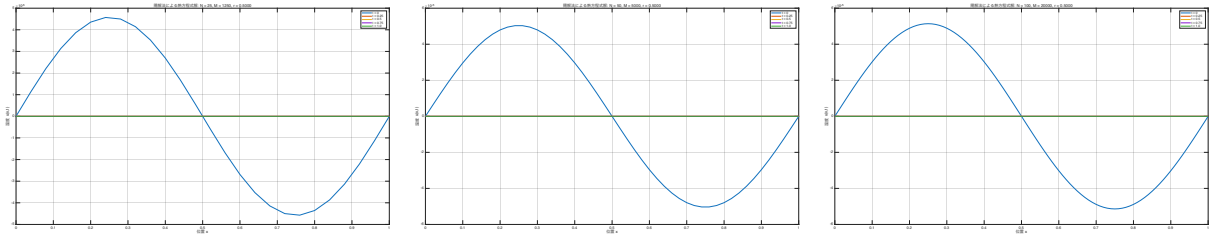
図 4: 各 (N, M) について，空間 x に対する誤差

図 3，4 に示されるように，空間分割数 N ，時間分割数 M を増加させることで，全体的な絶対誤差は減少する。また絶対誤差は高周波成分を含みつつも振幅が小さく，より局所的・複雑な構造を持つようになる傾向が見られる．

さらに，初期時刻 $t = 0$ において絶対誤差は最大となり，時間の経過とともにその振幅は減少していく．この挙動は拡散方程式の持つ平滑化作用によるものであり，解のなめらかさが増すにつれて絶対誤差も消散していくことを示している．

絶対誤差の概形は全体的に左右対称であり，これは初期条件 $f(x) = 1$，境界条件 $a = b = 0$ による対称性を反映している．

次に $f(x) = \sin(2\pi x)$ の場合について考察する．



$(N, M) = (25, 1250)$　　$(N, M) = (50, 5000)$　　$(N, M) = (100, 20000)$

図 5: 各 (N, M) に対する数値解 u の様子

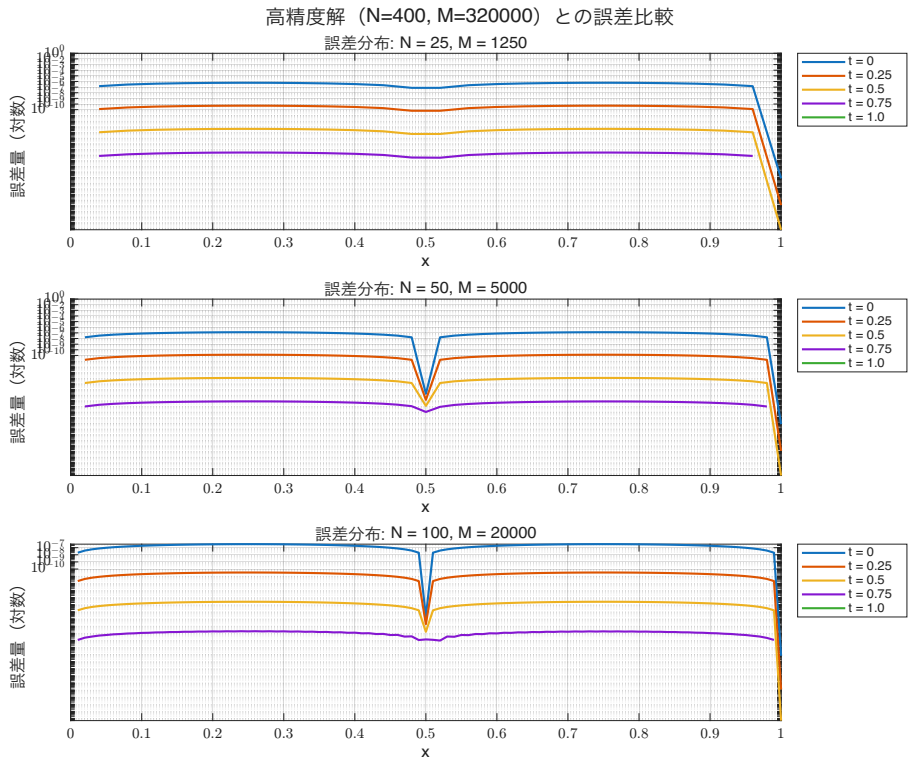


図 6: 各 (N, M) について，空間 x に対する絶対誤差

図 5 から，$f(x) = \sin(2\pi x)$ の時は，$t = 0$ を除き，数値解は非常に小さく，10^{-5} のオーダーである（$t = 0$ は初期状態）．また，解は急速に 0 に収束しているように見え，これは安定性パラメーター r が $r \leq 0.5$ を満たし，非常に安定していることと，初期条件 $f(x) = \sin(2\pi x)$ にある．$\sin(2\pi x)$ は周波数の高い関数であるため，$t = 0.25$ 時点で 10^{-17} のオーダーまで小さくなり，ほぼ 0 である．当然 t がこれより大きくなれば解は小さくなるので，$t = 0.25, 0.50, 0.75, 1.0$ の解は重なって見える．

図 6 から，誤差については，全体的にオーダーは 10^{-6} 程度であり，解のオーダーも 10^{-5} なので，相対誤差はやや大きいが t が $0.25 \sim 1.0$ のときは絶対誤差はほぼ無視できるほど小さい．特に，$t = 1.0$ の時は，グラフに現れないほど小さくなっている．また，分割数の増加，時間経過に伴って明らかに絶対誤差は小さくなっていて，空間精度の向上，時間の経過によって数値解が真の解に近づいていることがわかる．さらに絶対誤差は $x = 0.5$ 付近で最小になり，両端 $x = 0, 1.0$ 付近で大きくなる傾向がある．これは境界条件の影響や端点での離散化誤差が原因と考えられる．

ここで分割数による変化に注目してみる．分割数 N, M を上げても解の様子はあまり変化していないように見える．この原因を詳しく解析する．

まず解の精度は空間の分割幅 $\frac{1}{N}$ が小さくなればなるほど，即ち N が大きくなればなるほど上がる．また，上でも述べたように，初期条件 $\sin(2\pi x)$ は高周波な成分を持つ．この 2 点から，この初期条件では，$N = 50$ や $N = 100$ ではすでに見た目に明確な差が生まれるほどの誤差は残っていない．

別の原因としては，時間経過に伴って急激に解が減少するということが考えられる．この偏微分方程式の解析解は，

$$u(x, t) = \sin(2\pi x)\, e^{-4\pi^2 t}$$

であり，$e^{-4\pi^2 t}$ の t による値の変化は下の表のようになる．

t	$e^{-4\pi^2 t}$
0	1.00
0.25	3.0×10^{-5}
0.5	1.1×10^{-9}
0.75	3.5×10^{-14}
1.0	1.1×10^{-18}

$t = 1$ を見ると，10^{-18} のオーダーであり，ほぼ 0 である．そのため，分割数 N, M を増やしても解の大枠は変わらず，変化が見えづらいのである．

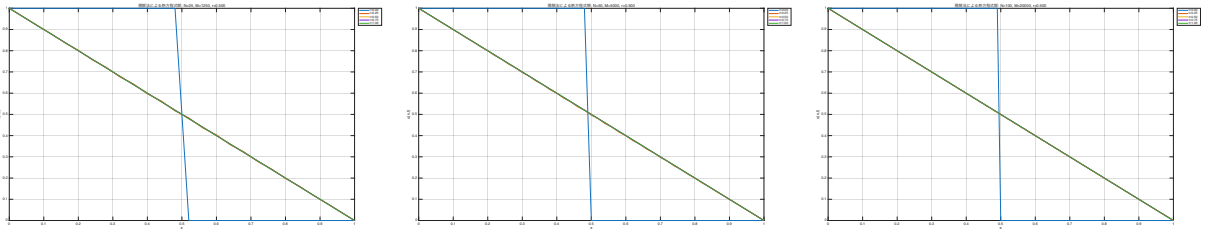
2.2.1 では，初期条件 $f(x)$ が $0 \leq x \leq 1$ で連続の場合を考えました．数値解の様子，誤差などについてわかったことを一度まとめておきたいと思います．
まず数値解の様子については，時間が経過すると，高周波成分は減衰し，解が滑らかになる平滑化現象が起こる．特に $t \geq 0.5$ の時は平坦に見え，$t = 0.5, 0.75, 1.0$ のプロットは重なって見える．
また、絶対誤差については，時間が経過するにつれて小さくなり，特に $t \geq 0.25$ の時はほぼ無視できるほど小さくなる．この傾向は初期条件が $f(x) = x - x^2$ や $f(x) = \sin(2\pi x)$ などの滑らかな関数に顕著に現れる．
最後に空間分割数 N，時間分割数 M の影響については，分割数を増やすことで絶対誤差は確実に小さくなっているが，初期条件 $f(x)$ が連続かつ滑らかな場合は，ある程度分割数が大きくなるとそれ以上精度は大きく向上しない．

2.2.2 初期条件 $f(x)$ が $0 \leq x \leq 1$ で連続でない場合

2.2.1 では初期条件 $f(x)$ が $0 \leq x \leq 1$ で連続な場合を考察したが，2.2.2 では $f(x)$ が $0 \leq x \leq 1$ で不連続な場合の解の様子，誤差伝播について考える．不連続な 初期条件 $f(x)$ として

$$f(x) = \begin{cases} 1 & (0 \leq x < 0.5) \\ 0 & (0.5 \leq x \leq 1) \end{cases}$$

を考える．ただし，拡散係数 $\alpha = 1$ とし，$a = b = 0$ とする．



$(N, M) = (25, 1250)$　　$(N, M) = (50, 5000)$　　$(N, M) = (100, 20000)$

図 7: 各 (N, M) に対する数値解 u の様子

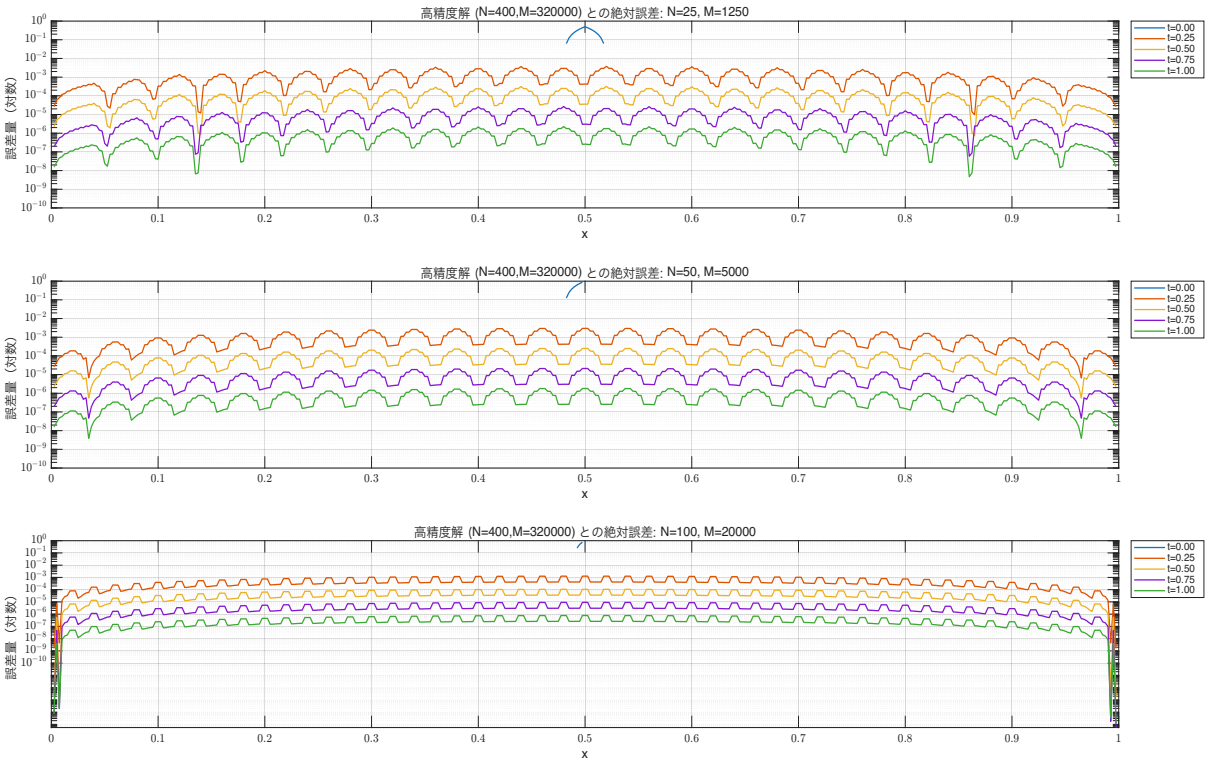


図 8: 各 (N, M) について，空間 x に対する絶対誤差

まず図 7 から，数値解の様子について考察する．$t = 0$ の時は，まだ計算をしていない状態なので分割数を上げようとも数値解は初期状態と全く同じである．時間が経過するにつれて不連続点 $(x = 0.5)$ を中心に両端へスプレッドし，振幅は指数的に減少し最終的には 0 付近に近づく．これは拡散方程式の本質である高周波成分の減衰と平滑化を反映している．

分割数に着目すると，分割数が少なく粗い場合は不連続面のスプレッドの幅が広く，値がなだらかに傾斜する領域が広い．分割数を大きくし，精度を上げると，スプレッドの幅が狭まり，傾斜がよりシャープになる．最も精度の高い $(N, M) = (100, 20000)$ では最も不連続面周辺の形状がわかりやすく，初期条件の輪郭が比較的はっきりと残っている．ただし，完全に振幅が 0 になることはなく，わずかに丸みが見える．

ここで，これらのことについて調べたことを書こうと思う．

まず不連続面の幅が縮まる理由を考える．

$t = 0$ では初期状態と同じ $x = 0.5$ を堺に値が変わる関数であるが，数値格子（時間格子）では，その境界をある幅だけぼかして表現している．このぼかし幅 Δ は，

$$\Delta \approx \sqrt{2\,\alpha\,\Delta t\,n}$$

のように，拡散係数 α とステップ数 n に応じて広がる．ここでステップ幅とは，時間方向の反復計算で何回更新したかを表す整数である．

$N = 100$ のとき，$\Delta x = \frac{1}{100}$ と十分細かいため，この数値的ぼかし幅が 1, 2 セル $(0.01 \sim 0.02)$ 程度に収まり，初期状態の輪郭が比較的シャープに見える．（セルとは分割した時の幅である）

次に $t = 0$ で丸みが見える原因を考える．はじめのステップ（更新）で，境界セルの隣のセルにも値が伝播す

る．差分法の更新式は，

$$U_j^{n+1} = U_j^n + r\left(U_{j+1}^n - 2U_j^n + U_{j-1}^n\right)$$

であり，第二項が隣接差を足す役割をしていて，ここでズレが生じている．安定条件 $r = \frac{\alpha \Delta t}{\Delta x^2} = 0.5$ の場合，この伝播は 1 ステップごとに隣のセルへ値の半分ずつ拡散するので，境界付近 $(x = 0.5)$ は左右 1 セルに少しずつ丸みがかかっている．
次に図 8 から，絶対誤差について考察する．$(N, M) = (400, 320000)$ の高精度解との絶対誤差は，常に $x = 0.5$ 付近で最大となる．これは不連続点において直前・直後に上振れまたは下振れをする Gibbs 現象や数値拡散の影響が最も強く出るためである．
分割数が増えると，絶対誤差は 10^{-1} のオーダーから 10^{-5} のオーダーまで小さくなる．また，時間経過に伴って絶対誤差は拡散，減衰する．初期直後の $t = 0.25$ で最も大きかった誤差が，t が 1 に近づくにつれて指数的に小さくなっていく．これは 2.2.1 でも述べた拡散方程式自体が数値誤差を消散させる性質を持つためである．

2.2.2 では初期条件 $f(x)$ が $0 \leq x \leq 1$ で連続でない場合について考察した．ここでわかったことをまとめておく．空間のステップ Δx や時間のステップ Δt の数値解や絶対誤差に対する影響は 2.2.1 の初期条件 $f(x)$ が $0 \leq x \leq 1$ で連続である場合と同じである．不連続関数特有のこととしては，不連続点周辺の挙動である．普通，不連続関数では，不連続点の直前にアンダーシュート（値が真値より下に振れる），直後にオーバーシュート（値が真値より上に振れる）という強い振動が現れる Gibbs 現象や，不連続点周辺の急な勾配が時間経過とともに丸く広がり，高周波成分が指数的に減衰することで，不連続点周辺が滑らかな山なりな形状になる拡散平滑化がせめぎ合っているが，陽解法を用いた差分法による解析では拡散平滑化が圧倒的に優勢である．前でも説明した通り，高周波成分をすぐに消散させる設計になっているため拡散平滑化が優勢なのです．また，不連続点を再現しようとすると現れる振動は不連続を完全に切り分けられないことに由来しますが，差分法は滑らかな関数を組み合わせて近似することはないため，そもそも振動的なオーバーシュートは起きません．つまり今回の場合は，Gibbs 現象のような振動は出ず，拡散によるぼかし（丸み）として現れているのです．

以上 2.2 では，初期条件が $0 \leq x \leq 1$ で連続かどうかによって分類して実験をし，初期条件 $f(x)$ が連続な場合は $f(x) = 1, x - x^2, \sin(2\pi x)$，連続でない場合は $f(x) = 1(0 \leq x < 0.5), 0(0.5 \leq x \leq 1)$ を扱った．これらの関数で実験した結果から，初期条件の連続性によって数値解，絶対誤差はどのように変わるのかを以下の点に着目してまとめる．

1. 解の滑らかさ
2. 数値拡散の現れ方
3. 分割数の増加による効果
4. 時間発展の見え方

1. 解の滑らかさ

連続初期条件はそもそも高周波成分が少ないため，誤差は小さく均一に減衰する．一方で不連続初期条件はジャンプ（不連続点の飛び）で高周波が入りやすく，そこに集中して大きな誤差を生じる．したがって連続関

数の方が滑らかに現れ，丸みではなく振幅減衰が起きる．

2. 数値拡散の現れ方

初期条件が連続の場合は高周波ノイズをあまり含まないため，数値拡散はどの成分も均等に緩やかに減衰させる作用として働く．そのため，指数的に振幅が小さくなり，形の歪みが小さくなる．絶対誤差も滑らかで，どの x でも同程度に小さく，時間とともに一様に減少する．一方で，不連続条件の場合は，更新ループの $1 \sim 2$ ステップで，グラフのジャンプ部の角がぼかされて丸みを帯びる．丸みは時間とともに拡散し，徐々に広い領域に広がった穏やかな山なり形状となる．絶対誤差は不連続点で最大となり，誤差が一箇所に集中し，そこから両端に向かって緩やかに小さくなる山なりのプロットとなる．丸みの幅は Δx のオーダーで決まり，分割数が小さいと広く大きくなり，大きいと鋭く小さくなる．また，連続の場合と同様に時間経過により精度は高くなり，誤差の最大値は小さくなる．

3. 分割数の増加による効果

連続条件では，波形のギザ付きや凹凸が減り，振幅が滑らかに落ちる曲線になっている．誤差は二次的 $(\propto \Delta x^2 = \frac{1}{N^2})$ に小さくなる．分割数 N, M を増やすとコストは $O(NM)$ で増加するが，精度は確実に向上する．不連続条件では，時間経過により急激に精度が向上するため，分割数の増加による形状の変化はあまり見られない．誤差のピーク幅は Δx に比例して丸み帯びるが，分割数を増やすと丸みが狭まり，より鋭い山なりの形状になる．全体としてジャンプ点以外は連続条件と同様，二次収束的に誤差が残り，ジャンプ点では丸みの影響で分割数の増加による効果が顕著に現れる．

4. 時間発展の見え方

連続条件では，振幅の減衰が主な変化である．山の高さは徐々に小さくなるが，波形の山と谷の位置，波長は一定である．グラフを重ねてみると，5 本の曲線が同じ山型を形を保ちつつ，時間経過とともに山がだんだん小さくなる．不連続条件の場合は，始めのステップですぐに収束し，$t = 0.25$ 付近にはステップは見られず，その後は中身が山のようなお椀型（波）になる．どちらの条件の場合も $t = 0.5$ 以降は全曲線がほぼ重なり，形の違いを目で追えないほどになる．

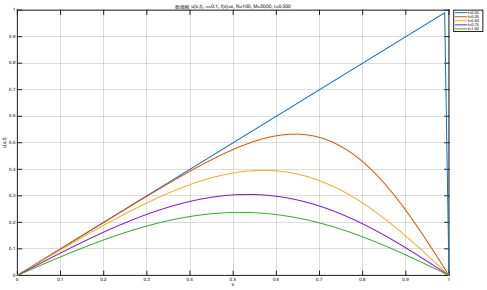
2.3 拡散係数 α による変化

2.2 では初期条件 $f(x)$ の違いによる数値解の振る舞いや誤差伝播の変化を見たが，2.3 ではさまざまな拡散係数 α について数値解，解析解との絶対誤差を見る．ただし，初期条件，境界条件は，$f(x) = x, a = b = 0$ とし，解析解はフーリエ正弦級数展開により求める．

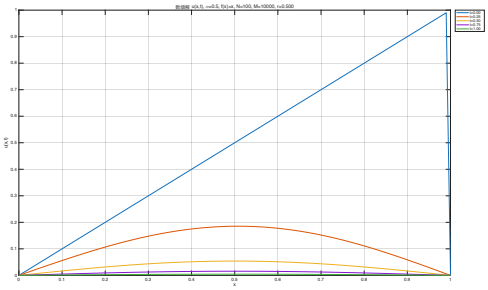
そもそも拡散係数 α は

$$\alpha = \frac{k}{\rho c_p}$$

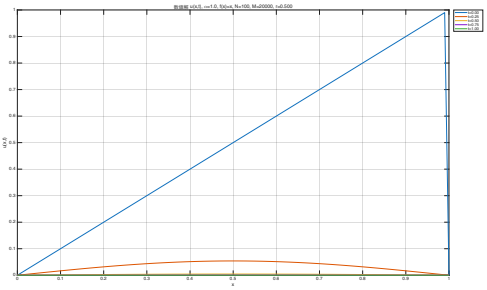
と定義され，熱伝導率，密度，比熱容量によって決まり，「熱を通す能力」/「熱を蓄える能力」となっている．α 大きいと良く熱を通し，小さいとあまり熱を通さない．つまり拡散係数は，時間変化の速さを決め，拡散の強さを表している．



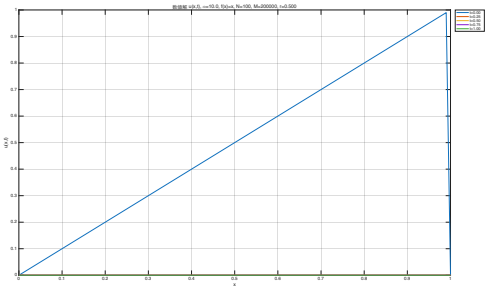
$\alpha = 0.1$



$\alpha = 0.5$



$\alpha = 1.0$



$\alpha = 10$

図 9: 各拡散係数 α に対する数値解 u の様子

図 9 より，拡散係数が小さい時（$\alpha = 0.1$），時間が進んでも初期状態の形状が残っており，$t = 1$ でも完全な平衡状態には達しない．拡散係数を少し大きくすると徐々に滑らかになり，t の増加にとともに中央部の温度は減衰する．空間全体に中程度の速度で広がっている様子が見られる．拡散係数を大きくすると，僅かな時間の経過で解が平坦になり，初期状態が一瞬で消え，非常に強い拡散によって全体が急速に均一化する．

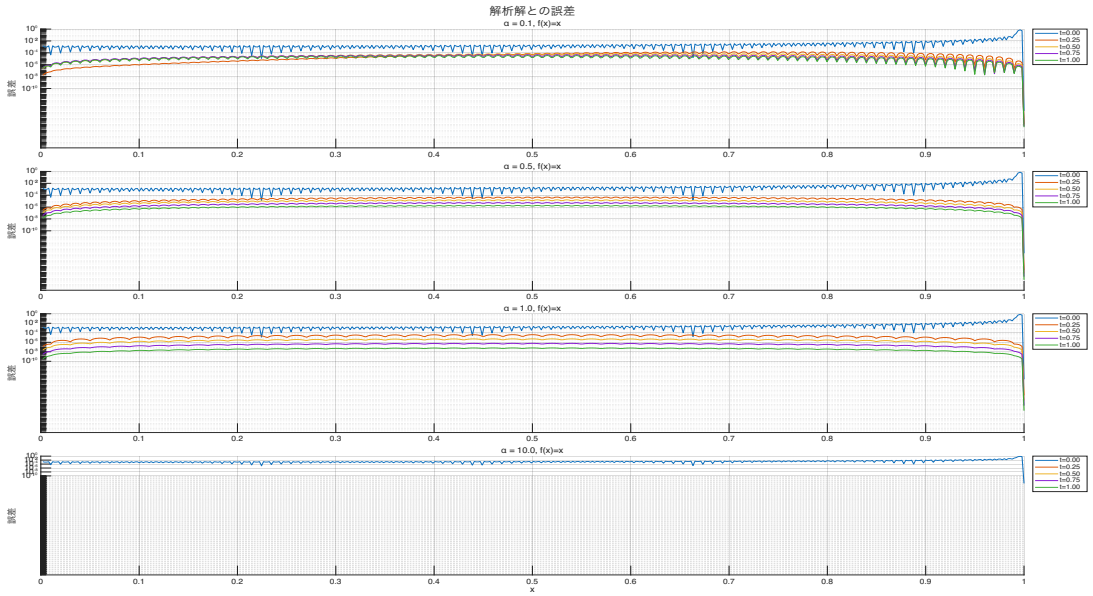


図 10: 各拡散係数 α に対する解析解との絶対誤差

図 10 より，拡散係数の小さい時（$\alpha = 0.1$）は，時間が経っても誤差が大きく，特に $x = 1$ で大きくなっている．拡散が弱く，数値拡散が抑えられている分，誤差が局所に現れている．拡散係数が増加していくと，誤差は全体的に滑らかになり，時間とともに綺麗に減少していく．拡散が進むことで，誤差が均されている．拡散係数を大きくすると，初期誤差はあるが，$t > 0$ では全ての時間で誤差はほぼ見えなくなる（マシンイプシロンレベルまで）．強い拡散により数値解は凄まじいスピードで解析解に近づく．
以上より，拡散係数が小さい場合は，解が急な勾配や鋭い形状を保つ時間が長いため，数値的に再現しづらく，数値解の近似の限界が顕著に現れる．拡散係数が大きい場合は，解が平坦化するため，誤差が急速に減少する．

2.3 では拡散係数を変えることによって考察した．ここでわかったことをまとめておく．
拡散係数は，大きいほど熱が速く全体に広がり（平坦），小さいと拡散がゆっくりで，初期状態を長く保つ傾向にある．そのため，誤差も拡散係数の増加に伴って小さくなっている．拡散係数が極端に大きい，小さい場合では，時間経過による拡散度合いが全く異なっており，それはグラフから明らかである．解く際の安定性については，安定条件の式から，拡散係数が大きいと，粗い格子でも解くことができるが，小さいと高解像度の細かい格子でないと解くことができない．実験より，拡散係数 α は単なるパラメーターではなく，時間発展の速さ・解の滑らかさ・数値解の安定性全てに大きく影響する物理量であることがわかった．

3　反復式について

このセクションでは，元の熱伝導方程式 $\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$ から条件を変えた時の反復式について考察する．

3.1　偏微分項の追加

まず数値的に解くために，格子（グリッド）を設定する．今までと同様，空間分割数を N，時間分割数を M とし，

$$\begin{aligned} x_j =& j\Delta x, \quad (j = 0, 1, \ldots, N) \\ t^n =& n\Delta t, \quad (n = 0, 1, \ldots, M) \end{aligned}$$

と設定する．すなわち，x_j は空間の j 番目の点，t^n は時間の n 番目の点である．
また，$u(x_j, t^n)$ における近似値を u_j^n とする．$(u(x_j, t^n) \approx u_j^n)$

3.1.1　元の熱伝導方程式について

まずは，これまで数値解を求めるために散々使ってきた元の熱伝導方程式

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$$

の反復式について見てみる．

時間微分（1 階）の前進差分は，

$$\frac{\partial u}{\partial t} \approx \frac{u_j^{n+1} - u_j^n}{\Delta t}$$

となり，

空間微分（2 階）の中心差分は，

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{j+1}^n - 2u_j^n + u_{j-1}^n}{\Delta x^2}$$

となる．これらを偏微分方程式に代入すると，

$$\begin{aligned}\frac{u_j^{n+1} - u_j^n}{\Delta t} =& \alpha \cdot \frac{u_{j+1}^n - 2u_j^n + u_{j-1}^n}{\Delta x^2} \\ u_j^{n+1} =& u_j^n + \frac{\alpha \Delta t}{\Delta x^2}\left(u_{j+1}^n - 2u_j^n + u_{j-1}^n\right)\end{aligned}$$

したがって反復式は，

$$\boxed{u_j^{n+1} = u_j^n + r\left(u_{j+1}^n - 2u_j^n + u_{j-1}^n\right)}$$

となる．ただし，$r = \frac{\alpha \Delta t}{\Delta x^2}$ であり，安定性パラメーターである．

$$r \leq \frac{1}{2} \Longleftrightarrow \Delta t \leq \frac{1}{2\alpha}\Delta x^2$$

陽解法はこの安定条件を満たしているとき，数値解が安定する．

3.1.2 偏微分項を加えた熱伝導方程式について

つぎは，1 階の空間微分項を加えた方程式

$$\frac{\partial u}{\partial t} + v\frac{\partial u}{\partial x} = \alpha \frac{\partial^2 u}{\partial x^2}$$

の反復式について考える．ただし，$v\frac{\partial u}{\partial x}$ は移流項である．
時間微分（1 階）の前進差分，空間 2 階微分（拡散項）は 3.1 と同じ．
空間 1 階微分（移流項）の上流差分は，

$$\frac{\partial u}{\partial x} \approx \frac{u_j^n - u_{j-1}^n}{\Delta x}$$

となる．時間微分，空間 1，2 階微分を偏微分方程式に代入すると，

$$\frac{u_j^{n+1} - u_j^n}{\Delta t} + v\frac{u_j^n - u_{j-1}^n}{\Delta x} = \frac{u_{j+1}^n - 2u_j^n + u_{j-1}^n}{\Delta x^2}$$

したがって反復式は，

$$\boxed{u_j^{n+1} = u_j^n - c\left(u_j^n - u_{j-1}^n\right) + r\left(u_{j+1}^n - 2u_j^n + u_{j-1}^n\right)}$$

ただし，$c = \frac{v \Delta t}{\Delta x}$，$r = \frac{\alpha \Delta t}{\Delta x^2}$ である．
安定条件としては，拡散係数 r は 3.1 と同じであり，移流項 c は $c \leq 1$ で安定する．

以上，反復式の違いを見てわかったことについてまとめる．
まず，項数の違いとしては，3.2 では移流項が追加され，時間による熱の広がりだけでなく，「運ばれる」という効果が追加されている．また，拡散は左右対称であるが，移流は方向を保つため，非対称な差分（上流差分）を用いる必要がある．さらに，安定性の観点から見ると，移流項の追加により，Courant 数 c の制限も加わるため，より慎重な安定化が必要である．
自然界では，時間経過により対称的に広がるものよりも，風や水の流れなど、移流的に広がるものの方が多い．そのため，自然現象の多くは 3.2 に対応している．

3.2 ノイマン境界条件

ここでは，ノイマン境界条件の下での反復式について考える．
まずノイマン境界条件とは，境界の微分（勾配）の値を指定する条件のことをいい，数学的な定義は，

$$\left.\frac{\partial u}{\partial x}\right|_{x=0} = g_0(t), \quad \left.\frac{\partial u}{\partial x}\right|_{x=L} = g_L(t)$$

であり，これは左端 $x=0$ と右端 $x=L$ での傾きがぞれぞれ，$g_0(t), g_L(t)$ と決まっている．
$g_0(t)=0$，$g_1(t)=0$ のとき，

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$$

の反復式ついて考える．
内部点 $j=1,2,\ldots,N-1$ については，

$$u_j^{n+1} = u_j^n + r\left(u_{j+1}^n - 2u_j^n + u_{j-1}^n\right)$$

となり変わらないため，端点について考える．

- **空間の左端について**
  条件は $\left.\frac{\partial u}{\partial x}\right|_{x_0=0} = 0$ であり，中心差分近似は，

  $$\frac{u_1^n - u_{-1}^n}{\Delta x} = 0 \implies u_{-1}^n = u_1^n$$

  となるから，反復式は，

  $$u_0^{n+1} = u_0^n + r\left(u_1^n - 2u_0^n + u_1^n\right) = u_0^n + 2r\left(u_1^n - u_0^n\right)$$

  となる．

- **空間の右端について**
  条件は $\left.\frac{\partial u}{\partial x}\right|_{x_N=1} = 0$ であり，中心差分近似は，

  $$\frac{u_{N+1}^n - u_{N-1}^n}{\Delta x} = 0 \implies u_{N+1}^n = u_{N-1}^n$$

  となるから，反復式は，

  $$u_N^{n+1} = u_N^n + r\left(u_{N-1}^n - 2u_N^n + u_{N-1}^n\right) = u_N^n + 2r\left(u_{N-1}^n - u_N^n\right)$$

  となる．

したがって，ノイマン境界条件の下での反復式は，

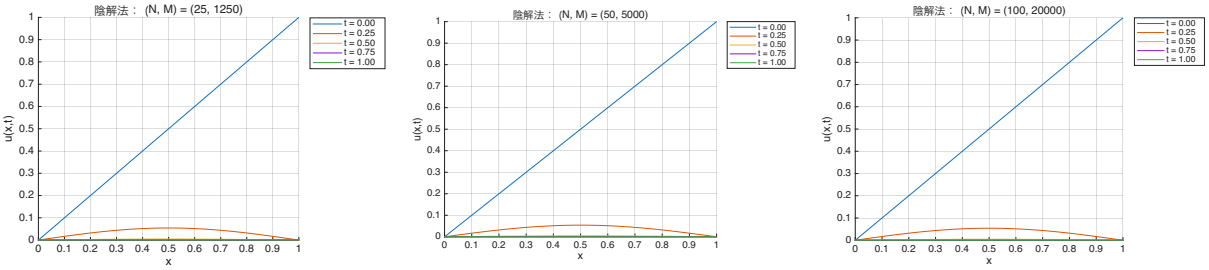
$$\begin{cases} u_0^{n+1} = u_0^n + 2r\left(u_1^n - u_0^n\right) & (j=0) \\ u_j^{n+1} = u_j^n + r\left(u_{j+1}^n - 2u_j^n + u_{j-1}^n\right) & (1 \leq j \leq N-1) \\ u_N^{n+1} = u_N^n + 2r\left(u_{N-1}^n - u_N^n\right) & (j=N) \end{cases}$$

となる．

ノイマン境界条件の特徴は，Dirichlet 条件では表現できない断熱などが表現でき，仮想的な点 (u_{N-1}, u_{N+1}) を用いて差分法を正しく構成することができることである．そのため，自然境界条件を簡潔にモデル化でき、物理現象に即している．
しかし，Dirichlet 条件と違い，境界の値が決まるわけではないので，振動や数値誤差が入りやすく，安定性に欠けるなどのデメリットもある．

4　陰解法について

これまでは陽解法を用いて解析を行ってきたが，このセクションでは，陰解法を用いて解析を行う．陽解法との違いは，次の時刻を含む連立方程式を解くことにより解析を行う点である．連立方程式を解くため，計算コストは重いが，安定条件がないところが特徴である．
ここでは，初期条件 $f(x) = x$，境界条件 $a = b = 0$ の熱伝導方程式の陰解法による数値解，誤差伝播を陽解法と比較して考察する．



$(N, M) = (25, 1250)$　　$(N, M) = (50, 5000)$　　$(N, M) = (100, 20000)$

図 11: 各 (N, M) に対する数値解 u の様子

まず図 11 から，数値解の様子について考察する．時刻による変化は陽解法と同様で，時間経過に伴って解は 0 に近づいている．陽解法と異なる点は，分割数による変化である．陽解法は分割数が粗いとギザギザが残り，空間分割精度が向上するにつれて滑らかになる，一方で，陰解法は分割数による違いがほとんど現れず，無条件安定であるがはっきりとわかる．ここで分割数による変化の少なさや，解の安定性について調べたことをまとめる．
フォン・ノイマンの安定性解析を使って数式的に見る．まず，陰解法の反復式は，

$$u_j^{n+1} = u_j^n + r\left(u_{j+1}^{n+1} - 2u_j^{n+1} + u_{j-1}^{n+1}\right)$$

である．

解の誤差をフーリエモードで表すと，

$$u_j^n = \xi^n e^{ikj\Delta x}$$

となる．ただし，ξ は増幅係数，k は波数，i は虚数単位を表す．これを反復式に代入して ξ について解くと，

$$\xi = \frac{1}{1 + 4r\sin^2\left(\frac{k\Delta x}{2}\right)}$$

となる．増幅係数 ξ が $|\xi| \leq 1$ を満たす時，解は安定するが，この式よりそれは明らか．よって，全ての点で $|\xi| \leq 1$ を満たすため，無条件で安定するのである．ちなみに，陽解法の増幅係数は，

$$\xi = 1 - 4r\sin^2\left(\frac{k\Delta x}{2}\right)$$

となるため，安定条件 $r \leq \frac{1}{2}$ が導かれる．
以上の数式的な考察により，分割数によるグラフの概形の変化が少ないのは，粗い分割でも安定して解は収束するからであることがわかった．

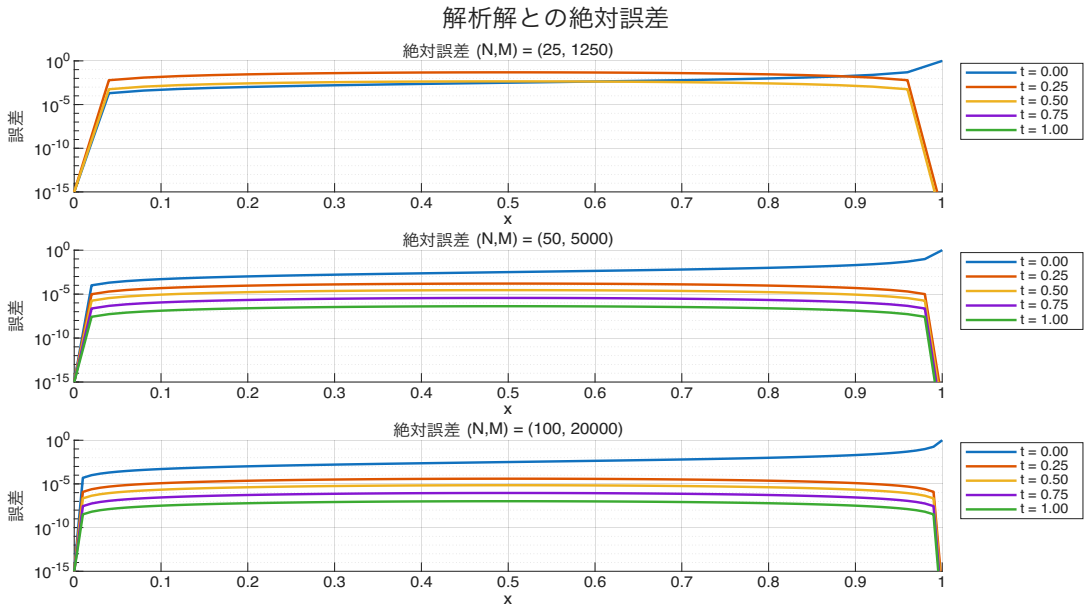


図 12: 各 (N, M) について，空間 x に対する絶対誤差

図 12 から，絶対誤差についても考察する．ただし，図 12 はフーリエ正弦級数展開による解析解との絶対誤差を表している．陽解法と同様に，時間経過により誤差は確実に小さくなっている．$t = 0$ では，解析解を有限和で打ち切ったことによる誤差が現れている．また，境界付近 $(x = 0, 1)$ ではギブス現象が起きており，Dirichlet 境界での近似誤差が現れている．陽解法と異なる点は，中央部分は横一線になっているところである．これは上でも見たように空間による誤差の変化が小さく，これは陰解法の安定性を表している．

陽解法と陰解法の違いをまとめておく．
反復法については，陽解法は一つ前の項から求めるのに対し，陰解法は求めたいものを含む連立方程式として計算するため，やや計算量が重い．

数値的な性質としては，陰解法は安定性に長けており，無条件で安定し，長時間計算でも安定して継続可能である．誤差についても，粗い分割でもしっかりと誤差が抑えられていて，緩い条件でもスムーズに収束する．総じて，陽解法は簡単に実装できるため，軽量な現象に向いていて，陰解法は高精度・長時間の安定解を得たい場合に向いていると言える．

5 まとめ

- **様々な条件の下での解析**
  初期条件や境界条件，パラメーターなど，様々な条件で解析を行った．対照実験により，それぞれが数値解の挙動や誤差伝播にどのような影響を与えるかが明確にわかった．さらに，定性的な考察に加え，定量的な考察することで，より詳しい内容も見た．定量的に見ることで，これまで習った関数の挙動がより正確に理解することができた．
- **数値解の安定性，精度と計算コストのバランス**
  安定性を維持するために Δt を小さくすると計算コストが増大する．一方精度向上のために Δx を小さくすると計算格子数が増え，こちらも計算コストが増大してしまう．そのため，解析する事象によって，陽解法，陰解法など，解析スキームを変えることが大切である．
- **グラフ可視化による理解の深化**
  $t = 0, 0.25, 0.5, 0.75, 1.0$ の各時刻で解の変化をグラフにプロットすることで，時間発展の様子を視覚的に把握することができた．特に，指数表記の Y 軸や補助線の追加など，誤差の変化が見やすくなる工夫を通じて，定性的・定量的な理解が深まった．
- **感想と展望**
  今回は人生で初めてこんなにも長いレポートを書いた．様々な数式やグラフの挿入はかなり難しく，苦しい時間が続いたが，上手く実装できた時の達成感や，満足感はとてもすごかった．本レポートを通して，陽解法による解析など数学的な理解も深まり，加えて，レポートの書き方や，説明の難しさなども体験できた．本レポートでは扱えなかった Crank-Nicolson 法などの解法や，グラフを 3 次元で表すことなどにも挑戦したみたいと思った．

6 参考文献

https://ocw.mit.edu/courses/18-336-numerical-methods-for-partial-differential-equations-spring-2009/
https://www.mathworks.com/help/pde/
https://www.latex-project.org/help/documentation/
Wikipedia：熱方程式，エネルギー保存則，フーリエ解析，有限差分法，ギブス減少，ノイマン境界条件，フォン・ノイマンの安定性解析

7 MATLAB のコード

```
function solution_by_alpha_linear()

    L = 1; T = 1; alpha = 1;
    ts = [0, 0.25, 0.5, 0.75, 1.0];
```

```
    test_configs = [25, 1250;
                    50, 5000;
                   100, 20000];

    for i = 1:3
        N = test_configs(i, 1);
        M = test_configs(i, 2);
        [x, u_result] = solve_heat_constant_IC(N, M, ts, alpha);


        figure('Position', [100 + 500 * (i - 1), 200, 500, 400]);
        plot(x, u_result, 'LineWidth', 1.5);
        grid on;
        xlabel('x'); ylabel('u(x,t)');
        title(sprintf('陽解法による熱方程式解(N=%d, M=%d, r=%.4f)', ...
            N, M, alpha * (T/M) / (L/N)^2), ...
            'FontSize', 12);
        legend(arrayfun(@(t) sprintf('t = %.2f', t), ts, 'UniformOutput', false), ...
               'Location', 'northeastoutside');
        xlim([0 1]); ylim([0 1]);
        xticks(0:0.1:1); xtickangle(0);
    end
end

function [x, result] = solve_heat_constant_IC(N, M, ts, alpha)
    L = 1; T = 1;
    dx = L / N;
    dt = T / M;
    r = alpha * dt / dx^2;

    x = linspace(0, L, N+1)';
    u = ones(N+1, 1);
    u(1) = 0; u(end) = 0;

    result = zeros(N+1, length(ts));
    current_time = 0;
    idx = 1;

    if ts(1) == 0
        result(:, idx) = u;
        idx = idx + 1;
    end

    for n = 1:M
        u_new = u;
        for j = 2:N
            u_new(j) = u(j) + r * (u(j+1) - 2*u(j) + u(j-1));
        end
        u = u_new;
        current_time = current_time + dt;

        if idx <= length(ts) && current_time >= ts(idx) - 1e-10
            result(:, idx) = u;
            idx = idx + 1;
        end
    end
end
```

Listing 1: 図 1 のコード

```
function heat_error_f1_display_fixed()

    f = @(x) ones(size(x));


    alpha = 1;
    L = 1; T = 1.0;
    snapshot_times = [0, 0.25, 0.5, 0.75, 1.0];


    N_ref = 400; M_ref = 320000;
    [x_ref, U_ref] = compute_numerical(N_ref, M_ref, f, alpha, T, snapshot_times);


    Ns = [25, 50, 100];
    Ms = [1250, 5000, 20000];

    figure('Name', 'Figure1', 'NumberTitle', 'off');
    sgtitle('高精度解 (N=400, M) との誤差比較=320000', 'FontWeight');

    for k = 1:length(Ns)
        N = Ns(k); M = Ms(k);
        [x_num, U_num] = compute_numerical(N, M, f, alpha, T, snapshot_times);


        E = zeros(N+1, length(snapshot_times));
        for j = 1:length(snapshot_times)
            u_ref_interp = interp1(x_ref, U_ref(:,j), x_num, 'spline');
            E(:,j) = abs(U_num(:,j) - u_ref_interp);
        end

        subplot(length(Ns), 1, k);
        semilogy(x_num, E, 'LineWidth', 1.2);
        box on;
        legend('t = 0', 't = 0.25', 't = 0.5', 't = 0.75', 't = 1.0', ...
               'Location', 'bestoutside');
        xlabel('x');
        ylabel('誤差');
        title(sprintf('N = %d, M = %d', N, M));
        ylim([1e-10, 1e0]);
        yticks(10.^(-10:2:0));
        yticklabels(strcat("10", ["\UTF{207B}\UTF{00B9}\UTF{2070}","\UTF{207B}\UTF
            {2078}","\UTF{207B}\UTF{2076}","\UTF{207B}\UTF{2074}","\UTF{207B}\UTF{00B2
            }","\UTF{2070}"]));
        set(gca, 'FontSize', 10);
        grid on;
    end
end


function [x, U_snapshots] = compute_numerical(N, M, f, alpha, T, snapshot_times)
    dx = 1 / N;
    dt = T / M;
    r = alpha * dt / dx^2;
```

```
    x = linspace(0, 1, N+1)';
    u = f(x);
    u(1) = 0; u(end) = 0;

    U_snapshots = zeros(N+1, length(snapshot_times));
    snapshot_indices = round(snapshot_times * M);
    count = 1;

    if snapshot_indices(1) == 0
        U_snapshots(:, count) = u;
        count = count + 1;
    end

    for n = 1:M
        u_new = u;
        for j = 2:N
            u_new(j) = u(j) + r * (u(j+1) - 2*u(j) + u(j-1));
        end
        u = u_new;

        if ismember(n, snapshot_indices)
            U_snapshots(:, count) = u;
            count = count + 1;
        end
    end
end
```

Listing 2: 図 2 のコード

```
function heat_eq_forward_euler()


    alpha = 1;


    params = [25, 1250; 50, 5000; 100, 20000];


    for k = 1:size(params,1)
        N = params(k,1);
        M = params(k,2);
        dx = 1/N;
        dt = 1/M;
        x = linspace(0,1,N+1);
        r = alpha * dt / dx^2;


        if r > 0.5
            warning('r = %.3f > 0.5 -> 不安定の可能性あり (N=%d, M=%d)', r, N, M);
        end


        u = ones(1,N+1);

        u(1) = 0;
        u(end) = 0;
```

```
        snapshots = zeros(5, N+1);
        time_indices = round(linspace(0, M, 5));
        snap_id = 1;


        for n = 1:M
            u_new = u;
            for i = 2:N
                u_new(i) = u(i) + r * (u(i+1) - 2*u(i) + u(i-1));
            end
            u = u_new;


            if ismember(n, time_indices)
                snapshots(snap_id,:) = u;
                snap_id = snap_id + 1;
            end
        end


        figure;
        hold on;
        colors = lines(5);
        for i = 1:5
            plot(x, snapshots(i,:), 'DisplayName', sprintf('t = %.2f', time_indices(i)*
                dt), 'Color', colors(i,:));
        end
        xlabel('位置 x');
        ylabel('温度 u(x,t)');
        title(sprintf('陽解法による熱方程式解 (N=%d, M=%d, r=%.3f)', N, M, r));
        legend('show');
        grid on;
        hold off;
    end
end
```

Listing 3: 図 3 のコード

```
function compare_with_reference()
    alpha = 1;
    T = 1.0;
    ts = [0, 0.25, 0.5, 0.75, 1.0];
    NM_list = [25, 1250; 50, 5000; 100, 20000];
    N_ref = 400; M_ref = 320000;


    [x_ref, U_ref_all] = compute_solution(N_ref, M_ref, ts, alpha, T);

    figure;

    for k = 1:size(NM_list,1)
        N = NM_list(k,1);
        M = NM_list(k,2);

        [x, U_all] = compute_solution(N, M, ts, alpha, T);
```

```
        E_all = zeros(length(ts), length(x_ref));
        for j = 1:length(ts)
            u_interp = interp1(x, U_all(j,:), x_ref, 'spline');
            E_all(j,:) = abs(u_interp - U_ref_all(j,:));
        end


        subplot(3,1,k)
        semilogy(x_ref, E_all.');
        ylim([1e-10 1e-1])
        ytickformat('10^%g')
        xlabel('x')
        ylabel('誤差量（対数）')
        title(sprintf('誤差分布: N = %d, M = %d', N, M))


        legend(arrayfun(@(t) sprintf('t = %.2f', t), ts, 'UniformOutput', false), ...
               'Location', 'northeastoutside')
    end

    sgtitle('高精度解 (N=400, M=320000) との誤差比較')
end


function [x, U_all] = compute_solution(N, M, ts, alpha, T)
    h = 1/N;
    dt = T/M;
    r = alpha * dt / h^2;

    x = linspace(0,1,N+1);


    u = ones(1, N+1);
    u(1) = 0; u(end) = 0;

    U_all = zeros(length(ts), N+1);
    t_index = 1;
    t_current = 0;

    U_all(t_index,:) = u;
    t_index = t_index + 1;

    for m = 1:M
        unew = u;
        for i = 2:N
            unew(i) = u(i) + r * (u(i+1) - 2*u(i) + u(i-1));
        end
        u = unew;
        t_current = m * dt;


        if t_index <= length(ts) && abs(t_current - ts(t_index)) < 1e-10
            U_all(t_index,:) = u;
            t_index = t_index + 1;
        end
```

```
    end
end
```

Listing 4: 図 4 のコード

```
function heat_explicit_solution()
    f = @(x) sin(2*pi*x);

    L = 1; T = 1.0;
    Ns = [25, 50, 100];
    Ms = [1250, 5000, 20000];

    for k = 1:length(Ns)
        N = Ns(k);
        M = Ms(k);

        dx = L / N;
        dt = T / M;
        r = dt / dx^2;

        x = linspace(0, L, N+1)';
        u = f(x);
        u_new = u;


        snapshot_times = [0, 0.25, 0.5, 0.75, 1.0];
        snapshot_idx = round(snapshot_times * M);

        solution = zeros(N+1, length(snapshot_times));
        count = 1;


        solution(:, count) = u;
        count = count + 1;

        for n = 1:M
            for j = 2:N
                u_new(j) = u(j) + r * (u(j+1) - 2*u(j) + u(j-1));
            end
            u = u_new;

            if ismember(n, snapshot_idx(2:end))
                solution(:, count) = u;
                count = count + 1;
            end
        end


        figure;
        plot(x, solution, 'LineWidth', 1.5);
        legend('t = 0', 't = 0.25', 't = 0.5', 't = 0.75', 't = 1.0');
        title(sprintf('陽解法による熱方程式解: N = %d, M = %d, r = %.4f', N, M, r));
        xlabel('位置 x'); ylabel('温度 u(x,t)');
        grid on;
    end
end
```

Listing 5: 図 5 のコード

```
function heat_explicit_error_plot()

    f = @(x) sin(2*pi*x);


    N_ref = 400; M_ref = 320000;
    L = 1; T = 1.0;
    dx_ref = L / N_ref;
    dt_ref = T / M_ref;
    alpha_ref = dt_ref / dx_ref^2;

    x_ref = linspace(0, L, N_ref+1)';
    u_ref = f(x_ref);
    u_ref_new = u_ref;

    snapshot_times = [0, 0.25, 0.5, 0.75, 1.0];
    snapshot_idx_ref = round(snapshot_times * M_ref);
    u_ref_snapshots = zeros(N_ref+1, length(snapshot_idx_ref));

    count = 1;
    for n = 1:M_ref
        for j = 2:N_ref
            u_ref_new(j) = u_ref(j) + alpha_ref * (u_ref(j+1) - 2*u_ref(j) + u_ref(j-1))
                ;
        end
        u_ref = u_ref_new;

        if ismember(n, snapshot_idx_ref)
            u_ref_snapshots(:, count) = u_ref;
            count = count + 1;
        end
    end


    Ns = [25, 50, 100];
    Ms = [1250, 5000, 20000];

    figure;
    sgtitle('高精度解 (N=400, M=32000)との誤差比較', 'FontSize', 14);

    for k = 1:length(Ns)
        N = Ns(k); M = Ms(k);
        dx = L / N;
        dt = T / M;
        alpha = dt / dx^2;

        x = linspace(0, L, N+1)';
        u = f(x);
        u_new = u;

        snapshot_idx = round(snapshot_times * M);
        error_data = zeros(N+1, length(snapshot_idx));

        count = 1;
```

```
        for n = 1:M
            for j = 2:N
                u_new(j) = u(j) + alpha * (u(j+1) - 2*u(j) + u(j-1));
            end
            u = u_new;

            if ismember(n, snapshot_idx)
                t = n * dt;
                u_ref_interp = interp1(x_ref, u_ref_snapshots(:, count), x, 'spline');
                error_data(:, count) = abs(u - u_ref_interp);
                count = count + 1;
            end
        end


        subplot(length(Ns), 1, k);
        semilogy(x, error_data, 'LineWidth', 1.5);
        title(sprintf('誤差分布: N = %d, M = %d', N, M));
        xlabel('x');
        ylabel('誤差量 (対数)');
        set(gca, 'YScale', 'log');
        yticks(10.^(-10:1:0));
        yticklabels({'$10^{-10}$','$10^{-9}$','$10^{-8}$','$10^{-7}$','$10^{-6}$','$10
            ^{-5}$','$10^{-4}$','$10^{-3}$','$10^{-2}$','$10^{-1}$','$10^{0}$'});
        set(gca, 'TickLabelInterpreter', 'latex');
        grid on;


        legend({'t = 0', 't = 0.25', 't = 0.5', 't = 0.75', 't = 1.0'}, ...
               'Location', 'northeastoutside');
    end


    set(gcf, 'Position', [100, 100, 1000, 800]);
end
```

Listing 6: 図 6 のコード

```
function heat_step_solution2()

    alpha = 1; L = 1; T = 1;
    Ns = [25,50,100];
    Ms = [1250,5000,20000];
    ts = [0,0.25,0.50,0.75,1.0];

    f = @(x) double(x<0.5);

    for k = 1:length(Ns)
        N = Ns(k); M = Ms(k);
        dx = L/N; dt = T/M; r = alpha*dt/dx^2;

        x = linspace(0,L,N+1)';
        u = f(x);
        u_new = u;


        snap_steps = round(ts * M);
```

```
        U = zeros(N+1, length(ts));
        cnt = 1;
        U(:,cnt) = u;  % ステップ0 → t=0
        cnt = cnt + 1;

        for n = 1:M
            for i = 2:N
                u_new(i) = u(i) + r*(u(i+1)-2*u(i)+u(i-1));
            end
            u = u_new;


            if cnt<=length(ts) && n==snap_steps(cnt)
                U(:,cnt) = u;
                cnt = cnt + 1;
            end
        end


        figure;
        plot(x, U, 'LineWidth',1.5);
        legend(arrayfun(@(t) sprintf('t=%.2f',t), ts,'Uni',false), ...
               'Location','northeastoutside');
        title(sprintf('陽解法による熱方程式解: N=%d, M=%d, r=%.3f',N,M,r));
        xlabel('x'); ylabel('u(x,t)');
        grid on;
    end
end
```

Listing 7: 図 7 のコード

```
function heat_step_error3()

    f = @(x) double(x<0.5);


    N_ref = 400; M_ref = 320000;
    L = 1; T = 1.0;
    ts = [0,0.25,0.5,0.75,1.0];


    x_ref = linspace(0,L,N_ref+1)';
    u_ref = f(x_ref);
    dt_ref = T/M_ref; r_ref = dt_ref/(L/N_ref)^2;
    U_ref = zeros(N_ref+1, numel(ts));
    idx_ref = round(ts * M_ref) + 1;
    count = 1;
    U_ref(:,count) = u_ref; count=2;
    for n=1:M_ref
        u_new = u_ref;
        for i=2:N_ref, u_new(i) = u_ref(i) + r_ref*(u_ref(i+1)-2*u_ref(i)+u_ref(i-1));
            end
        u_ref = u_new;
        if any(n+1 == idx_ref)
            U_ref(:,count) = u_ref; count=count+1;
        end
```

```
    end


    Ns = [25,50,100];
    Ms = [1250,5000,20000];

    figure;
    for k=1:3
        N = Ns(k); M = Ms(k);
        x = linspace(0,L,N+1)';
        u = f(x);
        dt = T/M; r = dt/(L/N)^2;


        U_num = zeros(N+1, numel(ts));
        idx_num = round(ts * M) + 1;
        count = 1;
        U_num(:,count) = u; count=2;
        for n=1:M
            u_new = u;
            for i=2:N, u_new(i) = u(i) + r*(u(i+1)-2*u(i)+u(i-1)); end
            u = u_new;
            if any(n+1 == idx_num)
                U_num(:,count) = u; count=count+1;
            end
        end


        E = zeros(numel(ts), N_ref+1);
        for j=1:numel(ts)
            un_interp = interp1(x, U_num(:,j), x_ref, 'linear', 'extrap');
            E(j,:) = abs(U_ref(:,j).' - un_interp.');
        end


        subplot(3,1,k)
        semilogy(x_ref, E.', 'LineWidth',1.2);
        xlim([0 1]);
        xlabel('x'); ylabel('誤差量（対数）');
        title(sprintf('高精度解 (N=400,M=320000) との絶対誤差: N=%d, M=%d', N,M));
        legend(arrayfun(@(t)sprintf('t=%.2f',t), ts,'uni',false), 'Location','
            northeastoutside');


        set(gca,'YScale','log');
        yticks(10.^(-10:1:0));
        yticklabels(arrayfun(@(n)sprintf('$10^{%d}$',n), -10:0, 'uni',false));
        set(gca,'TickLabelInterpreter','latex');
        grid on;
    end
end
```

Listing 8: 図 8 のコード

```
function solution_by_alpha_linear()

    L = 1; T = 1; ts = [0,0.25,0.5,0.75,1];
```

```
    N = 100; x = linspace(0,L,N+1)';

    alphas = [0.1, 0.5, 1, 10];
    for a = alphas

        dx = L/N;
        dt = 0.5 * dx^2 / a;
        M  = round(T/dt);
        dt = T/M;
        r  = a*dt/dx^2;


        U = compute_snapshots_linear(N, M, ts, a, L, T);


        figure;
        plot(x, U, 'LineWidth',1.5);
        legend(arrayfun(@(t) sprintf('t=%.2f',t), ts,'Uni',false), ...
               'Location','northeastoutside');
        title(sprintf('数値
            解 u(x,t), \\alpha=%.1f, f(x)=x, N=%d, M=%d, r=%.3f', a,N,M,r));
        xlabel('x'); ylabel('u(x,t)'); grid on;
        xlim([0 1]);

        saveas(gcf, sprintf('solution_linear_alpha_%.1f.png', a));
    end
end

function U = compute_snapshots_linear(N,M,ts,alpha,L,T)
    dx = L/N; dt = T/M; r = alpha*dt/dx^2;
    x = linspace(0,L,N+1)';


    u = x;
    u(1)=0; u(end)=0;

    U = zeros(N+1, numel(ts));
    U(:,1) = u;
    next_i = 2; target = ts(next_i);

    for n = 1:M

        u_new = u;
        u_new(2:end-1) = u(2:end-1) + r*(u(3:end) - 2*u(2:end-1) + u(1:end-2));
        u = u_new;

        t = n*dt;
        if next_i<=numel(ts) && t >= target - 1e-12
            U(:,next_i) = u;
            next_i = next_i+1;
            if next_i<=numel(ts)
                target = ts(next_i);
            end
        end
    end
end
```

Listing 9: 図 9 のコード

```
function error_by_alpha_linear_ana()

    L = 1; T = 1;
    ts = [0, 0.25, 0.5, 0.75, 1.0];
    alphas = [0.1, 0.5, 1, 10];
    N = 100; M = 20000;
    xnum = linspace(0,L,N+1)';


    yt = -10:2:0;
    ytl = strcat("10", ["\UTF{207B}\UTF{00B9}\UTF{2070}","\UTF{207B}\UTF{2078}","\UTF
        {207B}\UTF{2076}","\UTF{207B}\UTF{2074}","\UTF{207B}\UTF{00B2}","\UTF{2070}"]);

    figure('Units','normalized','Position',[0.1 0.1 0.7 0.9]);
    tiledlayout(numel(alphas),1,'Padding','compact','TileSpacing','compact');

    for k = 1:numel(alphas)
        a = alphas(k);


        Xana = linspace(0,L,500)';
        Uana = compute_analytical(Xana, ts, a);


        Unum = compute_snapshots_linear(N, M, ts, a, L, T);


        ax = nexttile;
        hold(ax,'on');
        for j = 1:numel(ts)

            u_interp = interp1(xnum, Unum(:,j), Xana, 'linear');
            semilogy(Xana, abs(Uana(:,j)-u_interp), 'LineWidth',1.2);
        end
        hold(ax,'off');

        xlim([0 1]); grid(ax,'on');
        set(ax,'YScale','log');
        yticks(10.^yt);
        yticklabels(ytl);
        ylabel('誤差');

        title(sprintf('α = %.1f, f(x)=x', a), 'Interpreter','none');
        if k==numel(alphas)
            xlabel('x');
        end
        legend(arrayfun(@(t) sprintf('t=%.2f',t), ts,'Uni',false), ...
               'Location','northeastoutside');
    end

    sgtitle('解析解との誤差','FontSize',16);
end

function U = compute_analytical(x, ts, alpha)
    K = 200;
    U = zeros(numel(x), numel(ts));
    for j = 1:numel(ts)
```

```
        t = ts(j);
        sumu = zeros(size(x));
        for n = 1:K
            c = 2*(-1)^(n+1)/(n*pi);
            sumu = sumu + c * sin(n*pi*x) * exp(-n^2*pi^2*alpha*t);
        end
        U(:,j) = sumu;
    end
end

function U = compute_snapshots_linear(N, M, ts, alpha, L, T)
    dx = L/N; dt = T/M; r = alpha*dt/dx^2;
    x = linspace(0,L,N+1)';
    u = x; u(1)=0; u(end)=0;


    snapshot_idx = round(ts*M);
    snapshot_idx(1) = 1;
    snapshot_idx(end) = M;
    next_i = 2;
    U = zeros(N+1, numel(ts));
    U(:,1) = u;

    for n = 1:M

        u(2:end-1) = u(2:end-1) + r*(u(3:end)-2*u(2:end-1)+u(1:end-2));

        if next_i<=numel(ts) && n==snapshot_idx(next_i)
            U(:,next_i) = u;
            next_i = next_i + 1;
        end
    end
end
```

Listing 10: 図 10 のコード

```
function implicit_heat_solver_each_case_separate_figures()
    L = 1; T = 1; alpha = 1;
    cases = [25, 1250;
             50, 5000;
             100, 20000];

    output_times = [0, 0.25, 0.5, 0.75, 1.0];

    for idx = 1:size(cases,1)
        Nx = cases(idx,1);
        Nt = cases(idx,2);
        dx = L / Nx;
        dt = T / Nt;
        r = alpha * dt / dx^2;

        x = linspace(0, L, Nx+1)';
        t = linspace(0, T, Nt+1);

        u = x;
        N = Nx - 1;
```

```
            e = ones(N,1);
        A = diag((1 + 2*r)*e) - diag(r*e(1:end-1), 1) - diag(r*e(1:end-1), -1);
        [Lmat, Umat] = lu(A);

            output_indices = round(output_times / dt) + 1;
        U_save = zeros(Nx+1, length(output_times));
        save_count = 1;
        U_save(:,save_count) = u;

        for n = 1:Nt
            b = u(2:end-1);
            y = Lmat \ b;
            u_new = Umat \ y;
            u(2:end-1) = u_new;
            u(1) = 0; u(end) = 0;

            if ismember(n+1, output_indices)
                save_count = save_count + 1;
                U_save(:,save_count) = u;
            end
        end

            figure('Position', [200, 200, 1000, 400]);
        hold on;
        for i = 1:length(output_times)
            plot(x, U_save(:,i), 'DisplayName', sprintf('t = %.2f', output_times(i)), '
                LineWidth', 1.0);
        end
        title(sprintf('陰解法： (N, M) = (%d, %d)', Nx, Nt), 'FontSize', 12);
        xlabel('x'); ylabel('u(x,t)');
        xticks(0:0.1:1);
        xtickangle(0);
        grid on;
        legend('Location', 'northeastoutside');
    end
end
```

Listing 11: 図 11 のコード

```
function implicit_heat_solver_error_plot_logscale_fixzero()
    L = 1; T = 1; alpha = 1;
    cases = [25, 1250;
             50, 5000;
             100, 20000];

    output_times = [0, 0.25, 0.5, 0.75, 1.0];
    num_terms = 100;
    tol = 1e-10;


    figure('Position', [200, 200, 800, 1000]);

    for idx = 1:size(cases, 1)
        Nx = cases(idx, 1);
        Nt = cases(idx, 2);
        dx = L / Nx;
        dt = T / Nt;
```

```
        r = alpha * dt / dx^2;

        x = linspace(0, L, Nx+1)';
        t_list = linspace(0, T, Nt+1);
        N = Nx - 1;
        u = x;

        e = ones(N,1);
        A = diag((1 + 2*r)*e) - diag(r*e(1:end-1), 1) - diag(r*e(1:end-1), -1);
        [Lmat, Umat] = lu(A);

         U_save = NaN(Nx+1, length(output_times));
        save_count = 1;
        U_save(:, save_count) = u;

        for n = 1:Nt
            b = u(2:end-1);
            y = Lmat \ b;
            u_new = Umat \ y;
            u(2:end-1) = u_new;
            u(1) = 0; u(end) = 0;


            for k = 1:length(output_times)
                if abs(t_list(n+1) - output_times(k)) < tol
                    save_count = save_count + 1;
                    U_save(:, save_count) = u;
                    break;
                end
            end
        end


        subplot(3, 1, idx); hold on;
        for k = 1:length(output_times)
            t_k = output_times(k);
            u_exact = analytic_solution(x, t_k, num_terms);
            err = abs(U_save(:,k) - u_exact);

            err(err == 0) = 1e-15;

            plot(x, err, 'DisplayName', sprintf('t = %.2f', t_k), 'LineWidth', 1.5);
        end
        title(sprintf('絶対誤差 (N,M) = (%d, %d)', Nx, Nt));
        xlabel('x'); ylabel('誤差');
        set(gca, 'YScale', 'log');
        ylim([1e-15, 1e0]);
        grid on;
        xticks(0:0.1:1);
        xtickangle(0);
        legend('Location', 'northeastoutside');
    end

    sgtitle('解析解との絶対誤差');
end

function u = analytic_solution(x, t, N_terms)
```

```
    u = zeros(size(x));
    for n = 1:N_terms
        coef = 2 * (-1)^(n+1) / (n * pi);
        u = u + coef * sin(n * pi * x) * exp(-(n * pi)^2 * t);
    end
end
```

Listing 12: 図 12 のコード